
심화 · 나머지 레버

원칙 5 개로 다 못 줄인 비용은 어디 있나

캐시 미스 · 압축 · 컴팩션 · 톨 출력 · 라우팅 · 배치 · 사고 예산

견적서의 나머지 항목

원칙 5 개로 다 못 줄인 비용은 어디 있나

캐싱 · 압축 · 컴팩션 · 서브에이전트 · 라우팅 · 배치

각 레버의 조건과 함정을 한 장씩 본다

레버는 곱해진다 — 우선순위는 뒤에서

캐시는 조건이 맞을 때만 이득

\$8.55 → \$3.219 (-62%)

프리픽스를 매번 바꾸면 \$10.05 (+17.5%)

고정 블록을 앞으로 모아 적중률을 올린다

같은 100콜, 캐시 조건만 바꿈 (Sonnet 4.6 단가)



압축과 캐싱은 대체재가 아니다

\$0.4014 → \$0.2007 (94% 절감)

그냥 캐싱 \$0.4014 vs 절반 요약 \$1.8000

지울 수 있는 반복은 지우고, 지울 수 없는 반복은 고정해 캐시에 태운다

압축률이 높을수록 조합의 이득도 커진다

긴 세션은 컴팩션으로 접는다

\$2.32 → \$0.93 (60% 절감)

10 턴마다 접으면 \$1.23 (47% 절감)

너무 자주 접으면 요약 호출이 이득을 먹는다

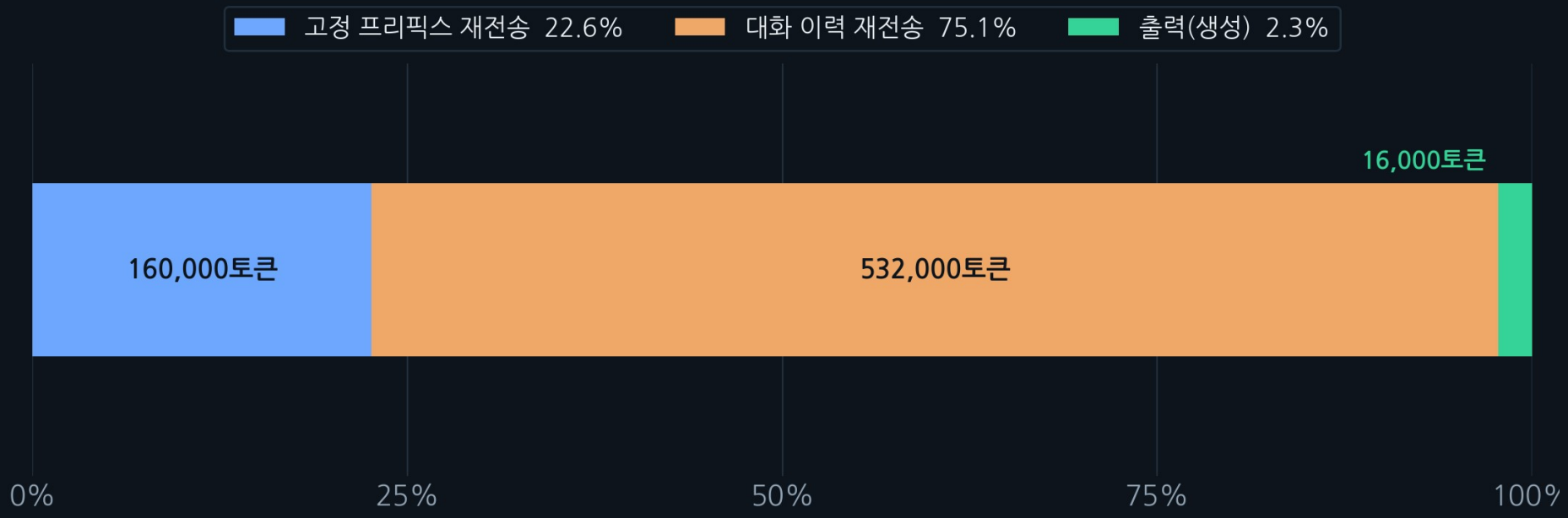
최적 주기는 세션 길이와 요약 비용으로 정해진다

세션 토큰의 4분의 3은 이력 재전송

고정 프리픽스 22.6% · 이력 75.1%

생성 (출력) 은 2.3% 뿐이다

그래서 ' 덜 쓰기 ' 보다 ' 다시 안 보내기 ' 가 크다

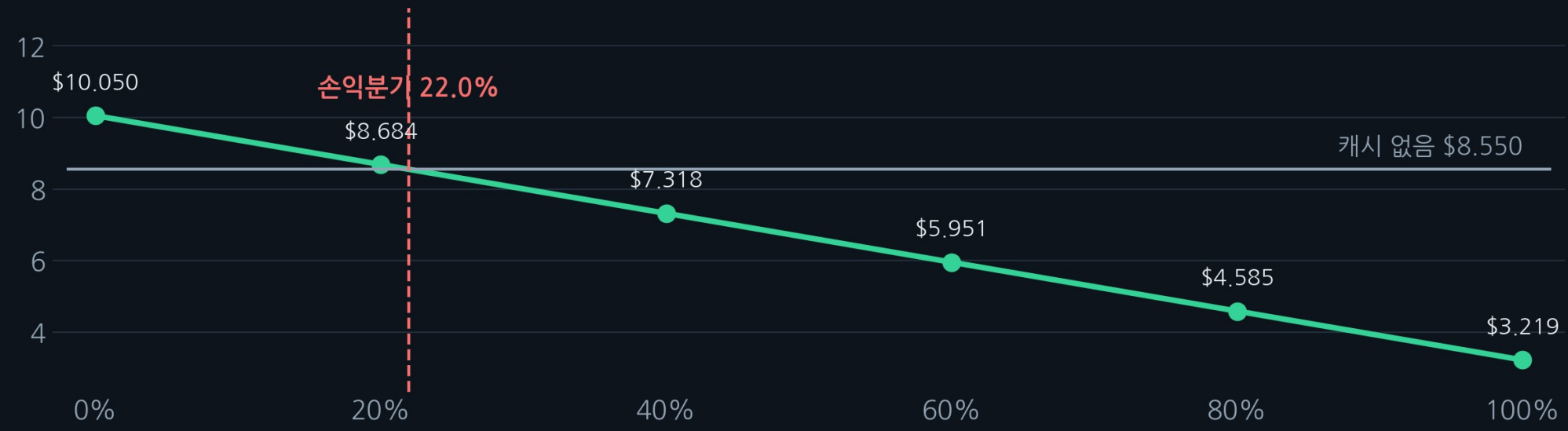


적중률 22% 면 본전

캐시 없음 \$8.550

적중률이 오를수록 단조 감소한다
미스가 섞이면 쓰기 프리미엄을 계속 문다

프롬프트 캐시 적중률에 따른 100콜 비용 (Sonnet 4.6)



툴 출력과 서버에이전트는 전체 비용으로 본다

\$5.6010 → \$2.7360

툴 읽기 누적 전송 1,680,000 토큰 (20 턴)

컴팩션 \$3.4425 · 서버에이전트 \$2.736

자식 호출 · 요약 비용까지 포함해 비교한다

모델 라우팅 · 쉬운 요청에 비싼 모델을 쓰지 않는다

전부 강 \$350 → 절반 \$184

절반만 라우팅해도 47% 절감

공격적 라우팅은 \$35.17 (90% 절감)

품질 측정 도구가 먼저다 — 임계값은 주기적으로 재보정

배치 API · 지연을 할인으로 바꾼다

50,000 건 \$675 → \$337.50

입력 · 출력 모두 50% 할인

24 시간 안에 끝나면 되는 잡만

야간 평가 · 대량 분류 · 리포트 생성 · 레포 전체 분석

작은 모델이 지휘한다

사다리 2.81 배 (1 턴 27% · 2 턴 43%)

실패가 반복되면 격차가 커진다 — 6 회 2.80 배

짧은 리워크에서도 이득 — 1 턴 27% · 2 턴 43%

respec 이면 실행 전에 멈추므로 실행 비용만 계산

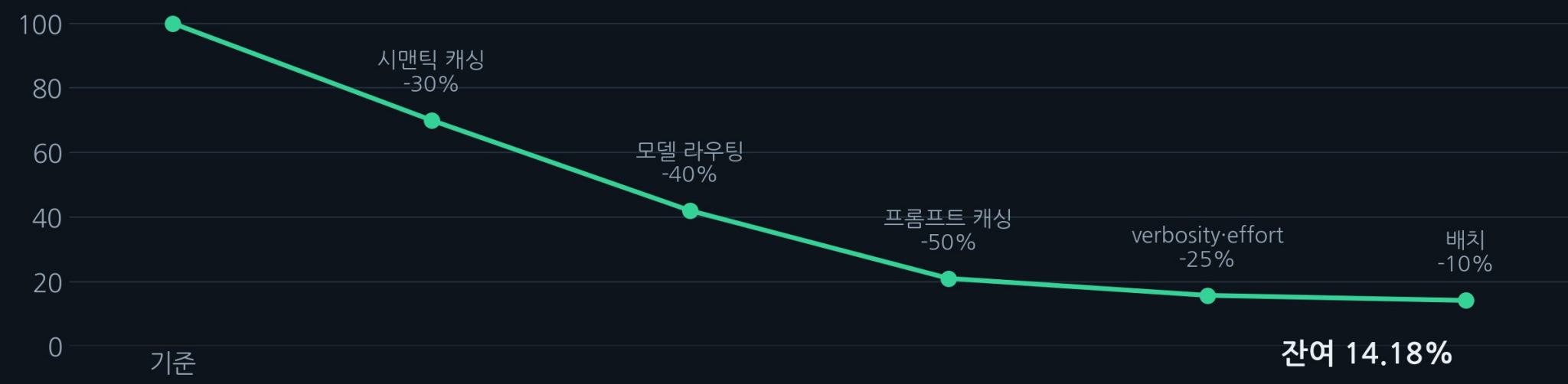
다섯 레버를 겹치면

잔여 14.18%

시맨틱 캐시 → 라우팅 → 캐싱 → effort → 배치

각 레버가 자기 조건을 만족할 때의 순차 산술

레버를 순서대로 적용한 잔여 비용 (%)



안 보이는 사고 토큰

199.47 배 (기존 환산 240.02 배)

같은 문제를 조건당 6 회 실행한 저장 로그 30 행

사고가 난 호출에서 사고 토큰 비중 99.625%

예산을 지정하지 않으면 사실상 ' 무제한 ' — 명시적으로 정한다

언어별 비용은 지표마다 다르다

호출당 1.364 배 · 0.729 배

문자당 (기존 환산 단가) 3.027 배 · 1.363 배

같은 과제라도 언어에 따라 생성 길이가 달라진다

지표를 섞으면 결론이 뒤집힌다

캐싱과 개인정보

개인화 · 실시간 응답은 캐싱 금지

멀티턴은 캐시 키에 컨텍스트를 포함

시맨틱 캐시 임계값이 느슨하면 '비슷하지만 틀린 답'을 준다

삭제 · TTL 정책을 프롬프트 설계와 함께 정한다

작게 시작해서 자기 usage 로 확인한다

오늘 한 가지만 켜라 — 캐시

다음 주에 배수와 적중률을 다시 켜다

품질 게이트를 함께 둔다 — 절감만으로 대신하지 않는다

질문과 재현 결과는 저장소 이슈로 남긴다